

4.6 用 VerilogHDL 描述组合逻辑电路

4.6.1 组合逻辑电路的行为级建模

4.6.2 分模块、分层次的电路设计

4.6.1 组合逻辑电路的行为级建模

组合逻辑电路的行为级描述一般使用 `assign` 结构和过程赋值语句、条件语句（`if-else`）、多路分支语句（`case-endcase`）和 `for` 循环语句等。

1、条件语句（if 语句）

条件语句就是根据判断条件是否成立，确定下一步的运算。

Verilog 语言中有 3 种形式的 if 语

包：(1) if (condition_expr)

true_statement;

(2) if (condition_expr) true_statement;

else false_statement;

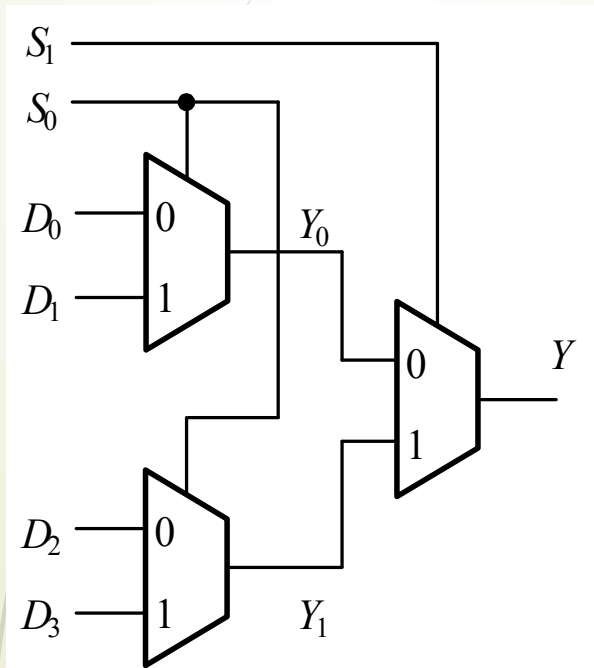
(3) if (condition_expr1) true_statement1;
else if (condition_expr2) true_statement2;
else if (condition_expr3) true_statement3;

.....

else default_statement;

if 后面的条件表达式一般为逻辑表达式或关系表达式。执行 if 语句时，首先计算表达式的值，若结果为 0、x 或 z，按“假”处理；若结果为 1，按“真”处理，并执行相应的语句。

例：使用 if-else 语句对 4 选 1 数据选择器的行为进行描述



```
module mux4to1_bh(  
    input [3:0] D ,  
    input [1:0] S ,  
    output reg Y  
) ; // 输入输出端口及变量数据类型
```

```
always @(D, S) // 电路功能描述  
    if (S == 2'b00)    Y = D[0];  
    else if (S== 2'b01) Y = D[1];  
    else if (S== 2'b10) Y = D[2];  
    else                Y = D[3];
```

```
endmodule
```

注意，过程赋值语句只能给寄存器型变量赋值，因此，输出变量 Y 的数据类型定义为 **reg**。

2、多路分支语句（case 语句）

是一种多分支条件选择语句，一般形式如下

```
case (case_expr)
    item_expr1: statement1;
    item_expr2: statement2;
    .....
    default: default_statement; //default 语句可以省略
endcase
```

注意：当分支项中的语句是多条语句，必须在最前面写上关键词 `begin`，在最后写上关键词 `end`，成为顺序语句块。

另外，用关键词 `casex` 和 `casez` 表示含有无关项 `x` 和高阻 `z` 的情况。

例：对具有使能端 En 的 4 选 1 数据选择器行为进行 Verilog 描述。当 $En=0$ 时，数据选择器工作， $En=1$ 时，禁止工作，输出为 0。

```
module mux4to1_bh(  
    input [3:0] D , [1:0] S, En,  
    output reg Y  
);  
    always @(D, S, En) //2001, 2005 syntax  
begin  
    if (En==1) Y = 0; //En=1 时，输出为 0  
    else          //En=0 时，选择器工作  
        case (S)  
            2'd0: Y = D[0];  
            2'd1: Y = D[1];  
            2'd2: Y = D[2];  
            2'd3: Y = D[3];  
        endcase  
    end  
endmodule
```

例：对基本的 4 线 -2 线优先编码器的行为进行 Verilog 描述。

```
module priority(  
    input [3:0] W,  
    output reg [1:0] Y  
);  
    always @(W)  
        case (W)  
            4'b1xxx: Y = 3;  
            4'b01xx: Y = 2;  
            4'b001x: Y = 1;  
            4'b0001: Y = 0;  
            default: Y = 2'bxx ; //W 无效时 ,Y 为高阻  
        endcase  
    endendmodule
```

3、for 循环语句

一般形式如下

```
for (initial_assignment; condition; step_assignment)  
statement ;
```

initial_assignment 为循环变量的初始值。

Condition 为循环的条件，若为真，执行过程赋值语句 **statement**，若不成立，循环结束，执行 **for** 后面的语句。

step_assignment 为循环变量的步长，每次迭代后，循环变量将增加或减少一个步长。

试用 Verilog 语言描述具有高电平使能的 3 线 -8 线译码器 .

```
module ecoder3to8_bh(  
    input [2:0] A ,  
    input En,  
    output reg [7:0]Y  
);  
    integer k;      // 声明一个整型变量 k  
    always @(A, En) //  
        begin  
            Y = 8'b1111_1111; // 设译码器输出的默认值  
            for(k = 0; k <= 7; k = k+1) // 下面的 if-else 语句循环 8  
                次  
                {  
                    if ((En==1) && (A== k) )  
                        Y[k] = 0; // 当 En=1 时, 根据 A 进行译码  
                    else  
                        Y[k] = 1; // 处理使能无效或输入无效的情况  
                }  
        end  
endmodule
```

例：用条件运算符描述了一个 2 选 1 的数据选择器。

```
module mux2x1_df (  
    input A,B,SEL;  
    output L  
);  
    assign L = SEL ? A :  
    B;
```

```
endmodule
```

在连续赋值语句中，如果 SEL = 1，则输出 L = A；否则 L = B。

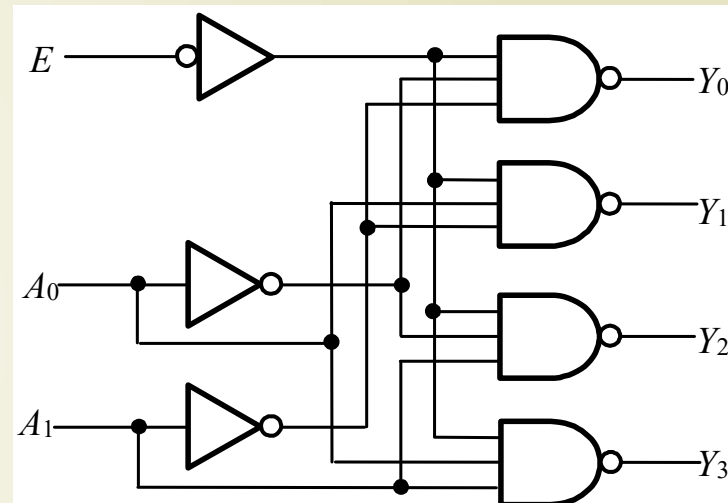
```
module mux2x1_df (  
    input A,B,SEL,  
    output reg Y  
);
```

```
    always @( D1,D0,S ) // 用 always 语句和条件运算符建
```

模

```
        L = S ? D1 : D0;  
endmodule
```

例：用数据流建模方法对
2 线 -4 线译码器的行为进
行描述。



```
module decoder_df (
    input A1,A0,E,
    output [3:0] Y
) ;
    assign Y[0] = ~(~A1 & ~A0 & ~E);
    assign Y[1] = ~(~A1 & A0 & ~E);
    assign Y[2] = ~(A1 & ~A0 & ~E);
    assign Y[3] = ~(A1 & A0 & ~E);
endmodule
```

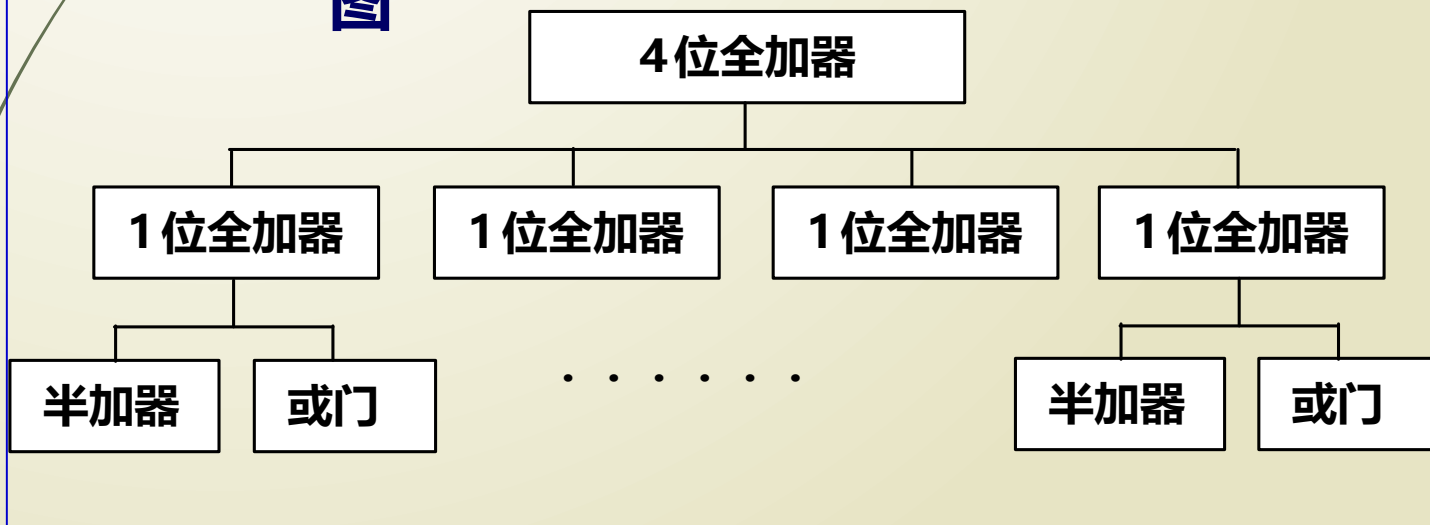
$$Y_0 = \overline{\overline{A_1} \overline{A_0} \overline{E}}$$
$$Y_1 = \overline{\overline{A_1} A_0 \overline{E}}$$
$$Y_2 = \overline{A_1 \overline{A_0} \overline{E}}$$
$$Y_3 = \overline{A_1 A_0 \overline{E}}$$

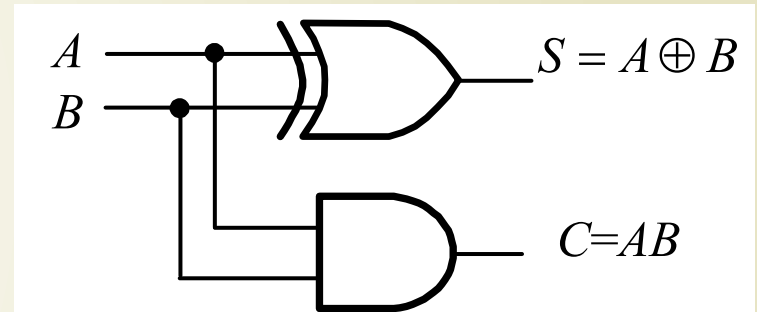
4.6.2 分模块、分层次的电路设计

分层次的电路设计：在电路设计中，将两个或多个模块组合起来描述电路逻辑功能的设计方法。

设计方法：自顶向下和自底向上两种常用的设计方法

4 位全加器的层次结构框图





```
module halfadder (  
    input A,B,  
    output S,C  
);  
    xor (S,A,B);  
    and (C,A,B);  
endmodule
```

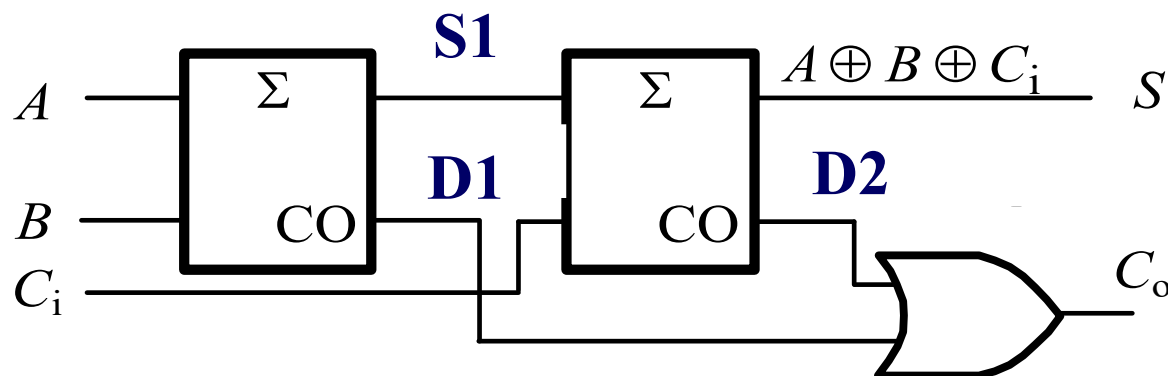
半加器的门级描述

```

module fulladder (
    input A,B,Ci,
    output S,CO
);
    wire S1,D1,D2; // 内部节点信号
    halfadder HA1 (.B(B),.S(S1),.C(D1),.A(A));
    halfadder HA2 (.A(S1),.B(Ci),.S(Sum),.C(D2));
    or (Co,D2,D1);
or g1(CO,D2,D1);
endmodule

```

全加器的描述
 - 名称关联调用半加器，括号外是原名称，括号里是现在名称，位置时序可改变。



```
module _4bit_adder (S,C3,A,B,C_1);
```

```
input [3:0] A,B;
```

```
input C_1;
```

```
output [3:0] S;
```

```
output C3;
```

```
wire C0,C1,C2; // 内部进位信号
```

```
fulladder FA0 (S[0],C0,A[0],B[0],C_1),
```

```
FA1 (S[1],C1,A[1],B[1],C0),
```

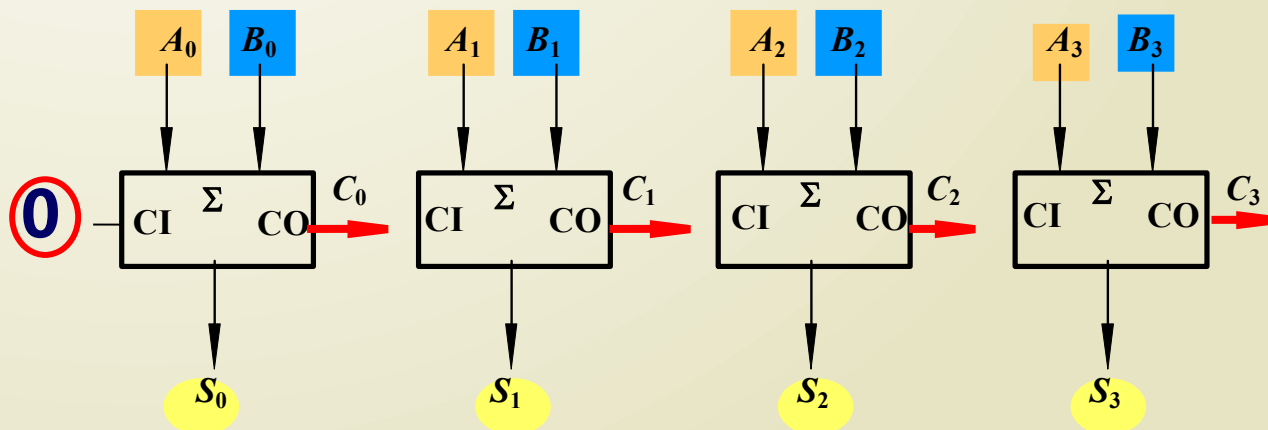
```
FA2 (S[2],C2,A[2],B[2],C1),
```

```
FA3 (S[3],C3,A[3],B[3],C2);
```

```
endmodule
```

4 位全加器的描述

-- 位置关联调用 1 位全加器，注意端口排列顺序



// 带参数的 4 位加法器子模块

module adder #(parameter n=4)

(

input [n-1:0] A,B,

input Ci,

output [n-1:0] S,

output Co

);

assign {Co,S} = A + B + Ci;

endmodule

} 4 位全加器的描述

// 按名称关联端口及参数

```
module adder_hier (  
    input [7:0] A1,B1,  
    input Cin1 ,  
    output [7:0] Sum1,  
    output Cout1
```

```
);  
    adder #( .n(8)) U1
```

```
(  
    .A(A1),  
    .B(B1),  
    .Ci(Cin1),  
    .S( Sum1),  
    .Co( Cout1)  
);
```

8 位全加器